

SISTEM MANAJEMEN PERPUSTAKAAN DIGITAL

Untuk Memenuhi Tugas Mata Kuliah Algoritma & Struktur Data

Dosen Pengampu:

Taufik Ridwan, M.T



Disusun Oleh: Kelompok 5

Fajar Raihan Hanafi	2510631250031
Lutfi Radithia Firmansyah	2510631250067
Muhamad Irfan Zainudin	2510631250070
Reihan Alfa Setiawan	2510631250043

**PRODI SISTEM INFORMASI
FAKULTAS ILMU KOMPUTER
UNIVERSITAS SINGAPERBANGSA KARAWANG**

2026

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi memberikan banyak kemudahan dalam pengolahan data, termasuk dalam bidang perpustakaan. Pengelolaan data buku secara manual sering menyebabkan berbagai kendala seperti proses pencarian yang lambat, data yang mudah hilang, serta kesalahan dalam pencatatan informasi buku. Oleh karena itu, diperlukan sebuah sistem digital yang mampu membantu pengelolaan data perpustakaan agar lebih terstruktur, cepat, dan efisien.

Program Sistem Perpustakaan Digital ini dibuat menggunakan bahasa pemrograman Java dengan memanfaatkan struktur data ArrayList untuk menyimpan data buku secara dinamis. Selain itu, program juga menerapkan konsep CRUD (Create, Read, Update, Delete), searching, sorting, dan file handling untuk menyimpan data secara permanen. Dengan adanya sistem ini, proses pengelolaan data buku dapat dilakukan dengan lebih mudah dan efektif.

1.2 Tujuan

Tujuan dari pembuatan program ini adalah:

1. Membuat sistem perpustakaan digital berbasis bahasa pemrograman Java.
2. Mengimplementasikan struktur data ArrayList dalam pengelolaan data buku.
3. Mengimplementasikan fitur CRUD pada sistem perpustakaan.
4. Mengimplementasikan algoritma searching dan sorting pada data buku.
5. Mengimplementasikan penyimpanan dan pembacaan data menggunakan file CSV.
6. Memahami penerapan pemrograman berorientasi objek (OOP) pada Java.
7. Menganalisis proses dan kompleksitas algoritma yang digunakan dalam program.

BAB II

ANALISIS KEBUTUHAN

2.1 Analisis Fitur Sistem

Program memiliki beberapa fitur utama, yaitu:

1. CRUD (Create, Read, Update, Delete)
 - a. Tambah data buku
 - b. Menampilkan data buku
 - c. Mengedit data buku
 - d. Menghapus data buku
2. Searching
 - a. Pencarian berdasarkan nama buku
 - b. Pencarian berdasarkan ID buku
 - c. Pencarian berdasarkan kategori
3. Sorting
 - a. Pengurutan berdasarkan ID
 - b. Pengurutan berdasarkan nama buku
 - c. Pengurutan berdasarkan jumlah buku
4. Additional Features
 - a. Statistik data perpustakaan
 - b. Menyimpan data ke file
 - c. Memuat data dari file
 - d. Menampilkan status sistem

2.2 Analisis Struktur Data

Program menggunakan struktur data ArrayList untuk menyimpan data buku.

Nama Array	Tipe Data	Fungsi
bukuList	List<Buku> / ArrayList<Buku>	Menyimpan seluruh data buku
nextId	int	Menyimpan ID otomatis berikutnya

id	int	Menyimpan ID buku
nama	String	Menyimpan nama buku
kategori	String	Menyimpan kategori buku
penulis	String	Menyimpan nama penulis buku
tahun terbit	int	Menyimpan tahun terbit buku
status	String	Menyimpan status buku (available, borrowed, deleted)
sc	Scanner	Menerima input dari pengguna

BAB III

IMPLEMENTASI

3.1 Struktur Data

Program terdiri dari class LibraryApp sebagai class utama dan class Buku sebagai representasi data buku. Sistem menggunakan struktur data ArrayList untuk menyimpan data buku secara dinamis. Navigasi program dilakukan melalui menu interaktif menggunakan perulangan do-while dan percabangan switch-case.

main()

- |— loadData()
- |— tambahBuku()
- |— tampilSemua()
- |— editBuku()
- |— hapusBuku()
- |— searchUnifikasi()
 - | |— search berdasarkan ID → binarySearch + bubble sort sementara
 - | |— search berdasarkan nama → linear search
 - | |— search berdasarkan kategori → linear search
- |— urutkanBuku()
 - | |— sortIdAsc() → bubble sort ascending
 - | |— sortNamaAsc() → selection sort ascending
 - | |— sortKategoriAsc() → selection sort ascending
- |— updateStatus()
- |— tampilStatistik()
- |— simpanData()
- |— idTerpakai()
- |— idLebihKecilDariIdTersimpanTerbesar()

3.2 Implementasi CRUD

tambahBuku() digunakan untuk menambahkan data buku baru ke dalam `ArrayList<Buku>`. Data buku dibuat menjadi objek `Buku` lalu dimasukkan ke `bukuList` menggunakan method `.add()`.

```
// Memasukkan objek buku baru ke dalam list dengan status default 'available'
bukuList.add(new Buku(id, nama, kat, penulis, tahun, status: "available"));
System.out.println("Buku ID " + id + " ditambah!");
}
```

hapusBuku() menggunakan metode soft delete, yaitu tidak menghapus objek dari `ArrayList`, melainkan hanya mengubah status buku menjadi "deleted".

```
// Fungsi untuk menghapus buku dengan metode "Soft Delete" (data tidak hilang dari sistem, hanya ganti status)
static void hapusBuku(Scanner sc) {
    System.out.print(s: "ID hapus: ");
    int id = sc.nextInt();
    sc.nextLine();

    // Mencari buku di database in-memory
    for (Buku b : bukuList) {
        if (b.id == id && !b.status.equals(anObject: "deleted")) {
            // Ubah status bukunya saja, data masih tersimpan untuk riwayat/statistik
            b.status = "deleted";
            System.out.println(x: "Dihapus soft.");
            return;
        }
    }
    System.out.println(x: "Tidak ditemukan.");
}
```

editBuku() mencari data buku berdasarkan ID menggunakan linear search, kemudian mengganti isi atribut buku secara langsung.

```
// Pencarian buku (Linear Search) berdasarkan ID yang diinputkan
for (Buku b : bukuList) {
    if (b.id == id && !b.status.equals(anObject: "deleted")) {
        // Meniban data lama dengan inputan data baru
        System.out.print(s: "Nama baru: ");
        b.nama = sc.nextLine();
        System.out.print(s: "Kategori baru: ");
        b.kategori = sc.nextLine();
        System.out.print(s: "Penulis baru: ");
        b.penulis = sc.nextLine();
        System.out.print(s: "Tahun baru: ");
        b.tahunTerbit = sc.nextInt();
        sc.nextLine();

        System.out.println(x: "Diedit!");
        return; // Menghentikan pencarian dan fungsi setelah berhasil edit
    }
}
```

3.3 Implementasi Searching

Linear Search (Nama & Kategori): Pencarian nama dan kategori dilakukan dengan menelusuri seluruh isi ArrayList satu per satu dan mencocokkan keyword menggunakan contains().

```
// Kriteria 2: Pencarian berdasarkan Nama (Linear Search / String matching)
} else if (tipe.equals(anObject: "nama")) {
    for (Buku b : bukuList) {
        // Ignore case (toLowerCase) dan mencocokkan kemiripan string pakai '.contains'
        if (!b.status.equals(anObject: "deleted") && b.nama.toLowerCase().contains(keyword.toLowerCase())) {
            System.out.printf(format: "ID %d: %s by %s (%s, %d)%n", b.id, b.nama, b.penulis, b.kategori, b.tahunTerbit);

            found = true;
        }
    }
}
```

Binary Search (ID): Pencarian ID menggunakan Binary Search setelah data diurutkan terlebih dahulu menggunakan Bubble Sort.

```
// Algoritma Binary Search untuk mencari buku secara efisien (membagi 2 rentang pencarian)
int low = 0, high = temp.size() - 1;
while (low <= high) {
    int mid = low + (high - low) / 2;
    if (temp.get(mid).id == idCari) { // Jika ditemukan di tengah
        Buku b = temp.get(mid);
        System.out.printf(format: "ID %d: %s by %s (%s, %d)%n", idCari, b.nama, b.penulis, b.kategori, b.tahunTerbit);

        found = true;
        break;
    } else if (temp.get(mid).id < idCari) low = mid + 1; // Geser batas pencarian bawah
    else high = mid - 1; // Geser batas pencarian atas
}
} catch (NumberFormatException e) {
    System.out.println(x: "ID angka!"); // Handling error jika input bukan tipe integer
}
```

3.4 Implementasi Sorting

Bubble Sort (ID Ascending): Bubble Sort digunakan untuk mengurutkan ID buku dari kecil ke besar.

```
// Algoritma pengurutan Bubble Sort secara Ascending (terkecil ke terbesar) berdasarkan integer ID
static void sortIdAsc() {
    for (int i = 0; i < bukuList.size() - 1; i++) {
        for (int j = 0; j < bukuList.size() - i - 1; j++) {
            // Pengecekan status agar indeks buku yang dihapus tidak ikut dibandingkan
            // Melakukan pertukaran (swap) jika objek sebelumnya lebih kecil angkanya
            if (!bukuList.get(j).status.equals(anObject: "deleted") && !bukuList.get(j + 1).status.equals(anObject: "deleted") &&
                bukuList.get(j).id > bukuList.get(j + 1).id) {
                Buku swap = bukuList.get(j);
                bukuList.set(j, bukuList.get(j + 1));
                bukuList.set(j + 1, swap);
            }
        }
    }
    System.out.println(x: "Urut ID asc (bubble).");
}
```

Selection Sort (Nama A-Z): Selection Sort digunakan untuk mengurutkan nama buku secara alfabetis.

```
// Algoritma pengurutan Selection Sort secara Ascending berdasarkan String nama buku
static void sortNamaAsc() {
    for (int i = 0; i < bukuList.size() - 1; i++) {
        int minIdx = i; // Menganggap elemen index 'i' adalah abjad terkecil (A paling kecil)
        for (int j = i + 1; j < bukuList.size(); j++) {
            // CompareToIgnoreCase akan menghasilkan angka negatif jika string kiri lebih kecil (berdasarkan urutan Lexicographic)
            if (!bukuList.get(j).status.equals(anObject: "deleted") && !bukuList.get(minIdx).status.equals(anObject: "deleted") &&
                bukuList.get(j).nama.compareToIgnoreCase(bukuList.get(minIdx).nama) < 0) {
                minIdx = j; // Update index buku dengan abjad nama terkecil
            }
        }
        // Jika ada nama dengan awalan abjad yang lebih kecil, maka elemen index 'i' di swap dengan elemen terkecil tsb.
        if (minIdx != i) {
            Buku swap = bukuList.get(i);
            bukuList.set(i, bukuList.get(minIdx));
            bukuList.set(minIdx, swap);
        }
    }
    System.out.println(x: "Urut nama asc (selection).");
}
```


Selection Sort (Kategori A-Z): Pengurutan kategori menggunakan konsep yang sama dengan pengurutan nama.

```
static void sortKategoriAsc() {  
    for (int i = 0; i < bukuList.size() - 1; i++) {  
        int minIdx = i;  
        for (int j = i + 1; j < bukuList.size(); j++) {  
            if (!bukuList.get(j).status.equals(anObject: "deleted") &&  
                !bukuList.get(minIdx).status.equals(anObject: "deleted") &&  
                bukuList.get(j).kategori.compareToIgnoreCase(bukuList.get(minIdx).kategori) < 0) {  
                minIdx = j;  
            }  
        }  
        if (minIdx != i) {  
            Buku swap = bukuList.get(i);  
            bukuList.set(i, bukuList.get(minIdx));  
            bukuList.set(minIdx, swap);  
        }  
    }  
}
```

3.5 Fitur Tambahan

Statistik: digunakan untuk menghitung jumlah total buku, jumlah buku tersedia (available), jumlah buku dipinjam (borrowed), dan jumlah buku yang dihapus (deleted). Perhitungan dilakukan dengan melakukan iterasi pada ArrayList<Buku>.

Simpan ke File: Penyimpanan data menggunakan PrintWriter dan FileWriter. Setiap objek buku pada ArrayList ditulis ke file CSV secara baris per baris.

Load dari File: Pembacaan data menggunakan BufferedReader dan FileReader. Setiap baris data dipisahkan menggunakan split(","), kemudian dikonversi kembali menjadi objek Buku dan dimasukkan ke dalam ArrayList.

BAB IV

ANALISIS KOMPLEKSITAS

4.1 Tabel Kompleksitas Waktu

No	Method/Operasi	Algoritma	Best Case	Average Case	Worst Case	Keterangan
1	tambahBuku()	Linear Scan & Push	$O(1)$	$O(n)$	$O(n)$	Didominasi pengecekan <i>idTerpakai</i>
2	tampilSemua()	Iterasi Penuh (For-each)	$O(n)$	$O(n)$	$O(n)$	Cetak semua buku jika status bukan deleted
3	editBuku()	Linear Search	$O(1)$	$O(n)$	$O(n)$	Berhenti di elemen pertama yang cocok untuk ditimpa
4	hapusBuku()	Linear Search (Soft Delete)	$O(1)$	$O(n)$	$O(n)$	Mengubah status menjadi deleted setelah ID ditemukan
5	searchUnifikasi() (Tipe: ID)	Bubble Sort + Binary Search	$O(n^2)$	$O(n^2)$	$O(n^2)$	Didominasi proses bubble sort pada list temp
6	searchUnifikasi() (Tipe: Nama/Kategori)	Linear Search (.contains)	$O(n)$	$O(n)$	$O(n)$	Scan seluruh array menggunakan <i>.contains()</i> tanpa <i>break</i>
7	sortIdAsc()	Bubble Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	Selalu dua loop penuh, membandingkan integer id
8	sortNamaAsc()	Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	Selalu dua loop penuh menggunakan

						<i>compareToIgnoreCase</i>
9	sortKategoriAsc()	Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	Sama dengan <i>sortNamaAsc</i> , mencari <i>minIdx</i> kategori
10	updateStatus()	Linear Search	$O(1)$	$O(n)$	$O(n)$	Didominasi pencarian ID sebelum mengubah status lewat ternary
11	tampilStatistik()	Iterasi Penuh	$O(n)$	$O(n)$	$O(n)$	Satu pass loop hitung total, available, dan borrowed
12	simpanData()	Iterasi + File I/O	$O(n)$	$O(n)$	$O(n)$	Iterasi panggil <i>toString()</i> untuk tulis n baris ke CSV
13	loadData()	Iterasi + File I/O	$O(n)$	$O(n)$	$O(n)$	Baca baris CSV + parsing objek + <i>Collections.shuffle</i>

4.2 Kompleksitas Ruang (Space Complexity)

Komponen	Kompleksitas	Keterangan
Struktur data utama (bukulist)	$O(n)$	Menggunakan Arraylist untuk menyimpan seluruh data objek buku secara dinamis
List objek sementara (temp)	$O(1)$	Salinan data sementara untuk kebutuhan pencarian ID.
Variabel lokal sorting (swap)	$O(1)$	variabel bantuan untuk pertukaran elemen.

Variabel lokal searching (low, high, mid)	$O(1)$	penanda binary (low, high, mid) bernilai tetap
Keseluruhan program	$O(n)$	Didominasi oleh penyimpanan struktur data utama dan list salinan sementara.

4.3 Penjelasan Detail

Kompleksitas searchUnifikasi() Berdasarkan ID tetap $O(n^2)$

Meskipun algoritma pencarian intinya menggunakan *Binary Search* yang secara teoritis beroperasi sangat efisien pada tingkat kompleksitas $O(\log N)$, fungsi pencarian ID ini mendahului proses pencarian dengan menyalin data ke list temporer dan memanggil algoritma Bubble Sort setiap kali dieksekusi. Oleh karena itu, performa totalnya sepenuhnya didominasi oleh proses pengurutan kuadratik tersebut. Efisiensi dari Binary Search yang berjalan di akhir proses menjadi tidak memberikan dampak signifikan terhadap performa waktu keseluruhan program. Solusi yang lebih optimal adalah menjaga data selalu terurut saat penambahan, atau menjalankan sort hanya sekali dengan menggunakan flag sudahTerurut.

Selection Sort dan Bubble Sort Selalu $O(n^2)$ Tanpa Pengecualian

Fungsi pengurutan berbasis Selection Sort (sortNamaAsc dan sortKategoriAsc) serta Bubble Sort (sortIdAsc) dalam program ini berjalan pada kompleksitas kuadratik $O(n^2)$ di setiap kondisi (best, average, maupun worst case). Hal ini terjadi karena algoritma diimplementasikan secara konvensional menggunakan dua kalang bersarang (nested loop) penuh tanpa adanya mekanisme early exit atau optimasi penanda (flag). Program akan tetap memindai dan membandingkan elemen berulang kali secara penuh, bahkan jika kondisi data masukan sebenarnya sudah terurut.

Kompleksitas tampilStatistik() Tetap Linear $O(n)$

Meski terdapat beberapa loop di dalam method ini, seluruh loop bersifat sekuensial dan tidak ada yang bersarang satu sama lain. Loop pertama menghitung total stok, maks, dan min dalam satu pass. Loop berikutnya mengiterasi 4 kategori, di mana setiap kategori menelusuri N data sehingga totalnya $4 \times N$ yang dalam notasi Big-O tetap disederhanakan menjadi $O(n)$

karena konstanta pengali diabaikan. Keseluruhan penggunaan memori tambahan (*space complexity*) pada operasi manipulasi lainnya juga tetap ditekan pada level konstan $O(1)$ karena mengadopsi sistem modifikasi data langsung di tempat.

BAB V

PENGUJIAN

Karena laporan ini dibuat dalam format teks, berikut disajikan simulasi output terminal untuk setiap skenario pengujian.

5.0 Jalankan Program

Input: Jalankan program *LibraryApp.java* untuk pertama kali melalui terminal

Output:

```
=== SISTEM MANAJEMEN PERPUSTAKAAN DIGITAL ===
1. Tambah buku
2. Tampil semua
3. Edit buku
4. Hapus (soft)
5. Search (id/nama/kategori)
6. Urutkan buku
7. Update status
8. Statistik
9. Simpan
0. Keluar
```

Hasil: Program berhasil dijalankan tanpa error dan sukses menampilkan 10 menu utama

5.1 Skenario 1 Tampil Semua

Input: Menu 2 (Tampilkan Semua)

Output:

Pilih: 2					
ID	Nama	Kategori	Penulis	Tahun	Status
28	Cerita Rakyat Nusantara	Cerita Rakyat	Kak Seto	2018	available
1	Jejak Langkah di Hutan Amazon	Petualangan	Ahmad Rifai	2018	available
26	Mengarungi Lautan Badai	Petualangan	Farah Quinn	2019	available
2	Kisah Sangkuriang	Cerita Rakyat	Siti Kretek	2021	available
19	Menembus Belantara Kalimantan	Petualangan	Tim Edukasi	2024	available
35	Detik-detik Proklamasi Agustus	Sejarah	Slamet Muljana	2014	available
21	Perang Diponegoro	Sejarah	Ir. Teguh	2016	available
16	Hikayat Hang Tuah	Cerita Rakyat	Lexie Xu	2018	available
8	Di Balik Runtuhnya Batara Guru	Sejarah	Budi Raharjo	2023	available
30	Tersesat di Labirin Kuno	Petualangan	Dr. Onno W.	2024	available
32	Gadis bangsawan di Nusantara	Sejarah	Ratih Kumala	2012	available
24	Bawang Merah dan Putih	Cerita Rakyat	Pratama Persada	2022	available
6	Legenda Batu Menangis	Cerita Rakyat	Raditya Dika	2020	available

Hasil: Berhasil, seluruh 30 data ditampilkan dalam format tabel.

5.2 Skenario 2 Tambah Buku

Input: Menu 1, ID buku: 36, Nama buku: Sang Lembah Ngawi, Kategori: Cerita Rakyat, Penulis: Novi hayati, Tahun terbit: 2004

Output:

```
Pilih: 1
ID buku: 36
Nama buku: Sang Lembah Ngawi
Kategori: Cerita Rakyat
Penulis: Novi Hayati
Tahun terbit: 2004
Buku ID 36 ditambah!
```

Hasil: Buku ke-36 berhasil disimpan, jumlah buku menjadi 36

```
Pilih: 1
ID buku: 5
ID telah terpakai, silakan coba dengan id lain
ID buku: █
```

5.3 Skenario 3 Edit Buku

Input: Menu 3, ID edit: 36, Nama baru: Goa darah jawa, Kategori baru: Cerita Rakyat, Penulis baru: Dadang Jaguar, Tahun baru: 2005

Output:

```
Pilih: 3
ID edit: 36
Nama baru: Goa darah jawa
Kategori baru: Cerita Rakyat
Penulis baru: Dadang Jaguar
Tahun baru: 2005
Diedit!
```

Hasil: Buku ke-36 berhasil disimpan, jumlah buku menjadi 36

5.4 Skenario Hapus data

Input: Menu 4, ID: 36

Output:

```
Pilih: 4
ID hapus: 36
Dihapus soft.
```

Hasil: Data berhasil dihapus secara soft delete.

5.5 Skenario Pencarian Data

Input: Pilih menu 5, pilih pencarian berdasarkan nama, lalu masukkan keyword “Legenda Batu Menangis”

Output:

```
Cari apa? (id/nama/kategori): id
Keyword/ID: 3
ID 3: Kala Rinjani Berselimut Awan by Tere Liye (Petualangan, 2019)
```

Hasil: Sistem menampilkan data buku sesuai keyword pencarian nama.

5.6 Skenario Pencarian Data Berdasarkan ID

Input: Pilih menu 5, pilih pencarian berdasarkan ID, lalu masukkan ID 3

Output:

```
Pilih: 5
Cari apa? (id/nama/kategori): nama
Keyword/ID: Legenda Batu Menangis
ID 6: Legenda Batu Menangis by Raditya Dika (Cerita Rakyat, 2020)
```

Hasil: Sistem menampilkan data buku sesuai ID yang dicari.

5.7 Skenario Pencarian Data Berdasarkan ID

Input: Pilih menu 6, lalu pilih opsi 1 (urut berdasarkan ID ascending)

Output: Data buku ditampilkan terurut berdasarkan ID dari kecil ke besar.

Pilih: 6
1. Urutkan berdasarkan ID (bubble asc)
2. Urutkan berdasarkan nama (selection asc)
3. Urutkan berdasarkan kategori (ascending A-Z)
Pilih: 1
Urut ID asc (bubble).

ID	Nama	Kategori	Penulis	Tahun	Status
1	Jejak Langkah di Hutan Amazon	Petualangan	Ahmad Rifai	2018	available
2	Kisah Sangkuriang	Cerita Rakyat	Siti Kretek	2021	available
3	Kala Rinjani Berselimut Awan	Petualangan	Tere Liye	2019	available
4	Sejarah Nusantara yang Hilang	Sejarah	Dr. Hendro	2015	available
5	Kidung Cinta di Majapahit	Romansa	Rian Setiawan	2022	available
6	Legenda Batu Menangis	Cerita Rakyat	Raditya Dika	2020	available
7	Penjelajah Samudra Hindia	Petualangan	Sri Utami	2017	available
8	Di Balik Runtuhnya Batara Guru	Sejarah	Budi Raharjo	2023	available
9	Melangkah Bersamamu di Badai	Romansa	Dee Lestari	2016	available

Hasil: Sistem berhasil mengurutkan data menggunakan metode Bubble Sort secara ascending berdasarkan ID.

5.8 Skenario Pengurutan Data Berdasarkan ID

Input: Pilih menu 6, lalu pilih opsi 2 (urut berdasarkan nama ascending)

Output:

Pilih: 6
1. Urutkan berdasarkan ID (bubble asc)
2. Urutkan berdasarkan nama (selection asc)
3. Urutkan berdasarkan kategori (ascending A-Z)
Pilih: 2
Urut nama asc (selection).

ID	Nama	Kategori	Penulis	Tahun	Status
13	Asal-Usul Selat Bali	Cerita Rakyat	Henry Ropeah	2019	available
24	Bawang Merah dan Putih	Cerita Rakyat	Pratama Persada	2022	available
17	Catatan Sejarah Revolusi Fisik	Sejarah	Yuki Tanaka	2022	available
28	Cerita Rakyat Nusantara	Cerita Rakyat	Kak Seto	2018	available
18	Cinta yang Tumbuh di Kota Tua	Romansa	Dr. Lukman	2023	available
35	Detik-detik Proklamasi Agustus	Sejarah	Slamet Muljana	2014	available
8	Di Balik Runtuhnya Batara Guru	Sejarah	Budi Raharjo	2023	available
10	Ekspedisi Kutub Utara	Petualangan	Prof. Gunawan	2014	available
32	Gadis bangsawan di Nusantara	Sejarah	Ratih Kumala	2012	available

Hasil: Sistem berhasil mengurutkan data menggunakan metode Selection Sort secara ascending berdasarkan nama.

5.9 Skenario Pengurutan Data Berdasarkan Kategori

Input: Pilih menu 6, lalu pilih opsi 3 (urut berdasarkan kategori ascending A–Z)

Output:

```
Pilih: 6
1. Urutkan berdasarkan ID (bubble asc)
2. Urutkan berdasarkan nama (selection asc)
3. Urutkan berdasarkan kategori (ascending A-Z)
Pilih: 3
Urut kategori asc (selection).
```

ID	Nama	Kategori	Penulis	Tahun	Status
13	Asal-Usul Selat Bali	Cerita Rakyat	Henry Ropeah	2019	available
24	Bawang Merah dan Putih	Cerita Rakyat	Pratama Persada	2022	available
28	Cerita Rakyat Nusantara	Cerita Rakyat	Kak Seto	2018	available
16	Hikayat Hang Tuah	Cerita Rakyat	Lexie Xu	2018	available
2	Kisah Sangkuriang	Cerita Rakyat	Siti Kretek	2021	available
6	Legenda Batu Menangis	Cerita Rakyat	Raditya Dika	2020	available
20	Legenda Candi Prambanan	Cerita Rakyat	Pandji Bagas	2017	available
33	Legenda Malin Kundang	Cerita Rakyat	dr. Ade Rai	2021	available
1	Jejak Langkah di Hutan Amazon	Petualangan	Ahmad Rifai	2018	available
3	Kala Rinjani Berselimut Awan	Petualangan	Tere Liye	2019	available
10	Ekspedisi Kutub Utara	Petualangan	Prof. Gunawan	2014	available
15	Memburu Harta Karun Sriwijaya	Petualangan	Maya Kartika	2020	available
19	Menembus Belantara Kalimantan	Petualangan	Tim Edukasi	2024	available

Hasil: Sistem berhasil mengurutkan data menggunakan metode Selection Sort secara ascending berdasarkan kategori.

5.10 Skenario Update Status Buku

Input: Pilih menu 7, masukkan ID 10, lalu pilih status 2 (borrowed)

Output:

```
Pilih: 7
ID: 10
1. available 2. borrowed
2
Status update!
```

Hasil: Status buku dengan ID 10 berhasil diperbarui menjadi borrowed.

5.11 Skenario Update Status Buku

Input: Pilih menu statistik

Output:

```
=== Statistik ===  
Total buku: 35  
Tersedia: 34  
Dipinjam: 1  
Deleted: 1
```

Hasil: Sistem berhasil menampilkan statistik data buku dengan total 35 buku, 34 tersedia, 1 dipinjam, dan 1 dihapus.

5.12 Skenario Menyimpan Data ke File CSV

Input: Pilih menu 9

Output:

```
Pilih: 9  
Disimpan ke data_buku.csv
```

Hasil: Sistem berhasil menyimpan seluruh data buku ke file **data_buku.csv**.

BAB VI

KESIMPULAN

6.1 Ringkasan

Program *LibraryApp* berhasil dibangun sebagai sistem manajemen perpustakaan digital berbasis CLI Java. Sistem ini mencakup modul utama yaitu CRUD, Searching, Sorting, Penyimpanan CSV dan Fitur Tambahan (Update Status dan Tampilkan Statistik) yang masing-masing telah diimplementasikan serta diuji.

Algoritma yang digunakan meliputi:

- Linear Search untuk pencarian nama/kategori, edit, hapus, dan update status dengan kompleksitas $O(n)$.
- Binary Search untuk pencarian ID dengan kompleksitas $O(\log n)$ di mana total proses menjadi $O(n^2)$ karena pengaruh pre-sort internal.
- Bubble Sort untuk pengurutan ID dengan kompleksitas $O(n^2)$.
- Selection Sort untuk pengurutan nama dan kategori dengan kompleksitas $O(n^2)$.

6.2 Kelebihan

- Boundary handling sudah mencakup: ID duplikat, input ID terlalu kecil, data kosong, data berstatus terhapus (*soft delete*), kerusakan kolom CSV, dan kesalahan input tipe data angka.
- Persistensi data berjalan baik dengan mekanisme simpan/load otomatis menggunakan file CSV agar data tidak hilang saat program ditutup.
- Struktur kode didesain modular melalui pembagian fungsi yang jelas sehingga program mudah dibaca, dirawat, dan dilacak saat terjadi *bug*.
- Pencarian teks sangat fleksibel berkat metode *.contains()* yang mendukung pencarian kata parsial dan bersifat case-insensitive.
- Perhitungan statistik sangat efisien karena mampu mengkalkulasi total, buku tersedia, dan buku dipinjam sekaligus dalam satu kali putaran loop.

6.3 Kelemahan Sistem dan Saran Perbaikan

Kelemahan Sistem	Saran Perbaikan
Pencarian ID memicu <i>Bubble Sort</i> yang mengubah urutan asli data di memori	Gunakan List baru khusus untuk proses sorting di dalam pencarian
Fungsi <i>Binary Search</i> tidak efisien karena data diacak saat load	Gunakan <i>Linear Search</i> biasa untuk mencari ID jika data sengaja diacak
Menu edit, hapus, dan update status akan crash jika diinput huruf	Bungkus input <i>sc.nextInt()</i> pada menu tersebut dengan blok try-catch
Input teks yang mengandung karakter koma (,) merusak baris file CSV	Tambahkan fungsi <i>.replace(",", " ")</i> pada input string sebelum disimpan
Data <i>soft delete</i> terus menumpuk dan membuat file CSV membengkak	Buat fungsi Purge untuk menghapus permanen data "deleted" secara berkala
Fungsi <i>updateStatus</i> menerima angka selain 1 dan 2 sebagai status <i>borrowed</i>	Tambahkan pengecekan ketat agar input yang diterima hanya angka 1 atau 2
Buku dengan nama dan penulis yang sama bisa dimasukkan berkali-kali	Tambahkan validasi kombinasi nama dan penulis sebelum buku baru disimpan

6.4 Penutup

Lewat program ini, fitur CRUD, pengelolaan data dengan ArrayList, sistem file CSV, hingga algoritma pencarian dan pengurutan terbukti bisa berjalan beriringan di Java. Meski masih ada beberapa kekurangan teknis yang harus diperbaiki, sistemnya sudah berfungsi lancar sesuai target awal dan siap dikembangkan lebih jauh menggunakan database relasional.